Smart Adaptive Traffic Management System for a Four-Way Junction

Chin Chun Yee
Faculty of Electrical Engineering
Universiti Teknologi Malaysia
Skudai 81310, Malaysia
chin.yee@graduate.utm.my

Soo Jiong Sin
Faculty of Electrical Engineering
Universiti Teknologi Malaysia
Skudai 81310, Malaysia
soosin@graduate.utm.my

Dev Verma
Faculty of Electrical Engineering
Universiti Teknologi Malaysia
Skudai 81310, Malaysia
devverma@graduate.utm.my

Abstract*—This paper presents the design, implementation, and bench validation of a low-cost smart adaptive traffic management prototype for a four-way junction. The system replaces a purely fixed-time signal sequence with an edge-computed controller based on an STM32 Nucleo-F446RE, eight HC-SR04 ultrasonic sensors, four pedestrian request buttons, and four red-yellow-green light modules. Each approach is represented by a near/far sensor pair that estimates traffic occupancy as no traffic, low traffic, or heavy traffic. The firmware executes a bounded circular phase order of north, east, south, and west while selecting green durations of 2 s, 5 s, or 10 s according to detected density. Safety is maintained using a 3 s yellow interval, a 2 s all-red gap, and interrupt-driven pedestrian requests that are served only during a safe red phase. Median-of-three ultrasonic readings, a noise floor, sensor settle delays, and Cortex-M4 DWT microsecond timing are used to improve reliability on a compact prototype. Test results show that the prototype achieved the intended adaptive timing behavior, non-starvation of approaches, deterministic pedestrian handling, and safe phase transitions. The discussion evaluates scalability, manufacturing cost, and commercialization requirements for an embedded smart-junction controller.

Keywords*—adaptive traffic control; STM32; ultrasonic sensor; embedded systems; pedestrian safety; four-way junction

I. Introduction

Urban intersections are a critical point of delay because several traffic streams compete for the same physical space. In a conventional fixed-time controller, the green interval allocated to an approach is usually selected during design or commissioning and remains unchanged during short-term changes in traffic demand. This makes the controller simple and predictable, but it also creates inefficiencies when one approach is empty while another contains a queue. The Federal Highway Administration distinguishes pre-timed operation from actuated operation by noting that actuated control responds to detector calls and extensions, while pre-timed control follows preset intervals [1]. The project presented in this paper adopts the same motivation at a prototype scale: it keeps the deterministic safety structure of a signal controller but replaces static green time with a compact sensor-driven decision.

The proposed system is a smart adaptive traffic management system for a four-way junction. It is implemented as an edge-computed embedded controller rather than a centralized cloud service. The prototype uses an STM32 Nucleo-F446RE board to acquire ultrasonic distance readings, handle external interrupts from pedestrian buttons, and drive red, yellow, and green LED modules. The hardware is intentionally modest. By avoiding cameras, high-performance processors, and network dependency, the design demonstrates how a small microcontroller can still deliver measurable improvements over a rigid timer in a classroom or small-junction context.

The design objective is not to replace certified municipal controllers directly. Instead, the prototype demonstrates a complete control methodology that can later be hardened into a commercial retrofit unit. The controller samples lane density, assigns bounded green intervals, maintains a circular service order to avoid starvation, and integrates pedestrian requests without abrupt interruption of active vehicle flow. These properties are valuable because practical signal controllers must balance throughput, fairness, safety, and cost.

II. Problem and Motivation

The project begins from a common bottleneck in urban traffic systems: fixed signal timing treats the intersection as if demand were approximately constant. When demand is balanced, fixed timing can provide a low-cost baseline. However, when one approach has heavy traffic and another approach is nearly empty, fixed timing wastes green time and increases queue delay. The project deck identified four related issues: lane starvation, economic drain from gridlock, environmental impact from excessive idling, and pedestrian disruption when button requests halt traffic flow abruptly. These issues motivate an adaptive controller that responds to local conditions while preserving safe signal order.

Lane starvation is especially relevant for a four-way junction because a naive priority policy can over-serve the busiest direction while preventing lightly loaded approaches from receiving green. Conversely, a purely cyclic fixed-time policy prevents starvation but still wastes time on empty roads. The system therefore uses a bounded circular sequence. Each approach receives service in the fixed order north, east, south, and west, but the length of the green phase changes based on detected density. This achieves fairness at the sequence level and responsiveness at the duration level.

Pedestrian requests introduce a second motivation. A pedestrian crossing phase should be safe and visible, but an immediate interruption of an active green can be disruptive and unsafe if vehicles are already moving through the junction. The prototype handles this by storing a pending request using an external interrupt and serving it during the corresponding direction's next red phase. This structure demonstrates a design philosophy that separates event detection from event execution. The button press is captured immediately, while the traffic state machine chooses a safe moment to respond.

A third motivation is cost. Many state-of-the-art adaptive signal systems rely on camera analytics, radar, vehicle-to-infrastructure communication, or networked optimization. These approaches are powerful, but they can be expensive,

sensitive to installation conditions, and difficult to justify for small deployments. This project explores a lower-cost path: ultrasonic distance sensors and a Cortex-M4 microcontroller produce a simple occupancy estimate that is sufficient for adaptive phase length selection.

III. Current Works

Traffic signal control has historically progressed from pre-timed control to actuated control, coordinated arterial systems, adaptive urban control systems, and intelligent learning-based methods. Pre-timed control is attractive because it is deterministic, simple to configure, and easy to validate. However, its performance decreases when traffic demand changes from the assumed timing plan. Actuated control improves this limitation by using detector calls and green extensions, allowing the controller to respond to vehicle presence and pedestrian requests [1], [2].

Large-scale adaptive systems such as SCOOT and SCATS established the principle that signal timing can be adjusted based on measured traffic demand. SCOOT was developed as a traffic-responsive method for coordinating signals in computerized urban traffic control networks [3]. These systems can improve traffic flow at the network level, but they normally require detector infrastructure, calibration, communication links, and continuous maintenance. Therefore, although they are powerful, they may be too complex or costly for small-scale junctions, campus roads, or low-budget deployments.

Research systems such as RHODES and SURTRAC further developed real-time scheduling and decentralized adaptive signal control. SURTRAC, for example, uses local scheduling and communication between intersections to support scalable urban traffic control [4]. This approach is relevant to the proposed project because it shows that traffic decisions do not always need to be made by a central server. Instead, each junction can perform local computation using real-time detector information.

Model predictive control has also been investigated for traffic signal control. In this approach, a traffic model is used to predict future system behavior, and the controller selects signal actions that optimize performance over a prediction horizon [11]. Although model predictive control can provide strong optimization capability, it usually requires an accurate traffic model and higher computational effort than a simple rule-based embedded controller.

More recent studies have explored reinforcement learning and deep reinforcement learning for adaptive traffic signal control. These methods can learn signal policies that reduce delay or queue length under simulated traffic conditions [5], [12]. A taxonomy of adaptive traffic signal control also shows that modern systems may be rule-based, optimization-based, cyclic, acyclic, local-only, or hierarchical [6]. However, learning-based systems introduce challenges such as explainability, safety verification, training-data requirements, and deployment reliability.

The proposed STM32-based prototype occupies a practical middle ground between fixed-time signal control and advanced network-level adaptive systems. It does not attempt to perform complex optimization or artificial-intelligence-based prediction. Instead, it uses a deterministic rule-based algorithm that maps ultrasonic sensor occupancy into three bounded green durations. This makes the system simple, low-cost, explainable, and suitable for prototype validation while still demonstrating the main advantage of adaptive traffic control: changing signal timing according to real-time traffic demand.

IV. Methodology

A. Overall System Design

The prototype is designed around a single STM32 Nucleo-F446RE board. The board provides an Arm Cortex-M4 microcontroller with sufficient GPIO resources, external interrupt support, and timing capability for ultrasonic ranging. The STM32F446 family provides a Cortex-M4 core with floating-point support and high clock frequency, while the Nucleo-64 board integrates ST-LINK programming and debugging, making it suitable for rapid prototyping [7], [8]. In the implemented design, the microcontroller performs all traffic decisions locally. It reads ultrasonic echo pins, stores pedestrian events, computes phase durations, and drives the LEDs that represent traffic signals.

Each traffic approach is represented by two HC-SR04 ultrasonic sensors: a near sensor and a far sensor. The near/far pair does not produce a full queue length in meters, but it allows the controller to classify an approach as empty, lightly occupied, or heavily occupied. This is a useful compromise for a prototype because it converts a continuous physical quantity into a small number of robust discrete states. The HC-SR04 module is commonly triggered by a short pulse and returns an echo pulse whose width corresponds to round-trip acoustic time; typical datasheets describe non-contact measurement over the centimeter-to-meter range and a 10 microsecond trigger requirement [10].

The output subsystem consists of four red-yellow-green LED modules, one for each direction. Twelve GPIO outputs drive the LEDs. This provides a visible demonstration of the complete signal sequence without requiring high-voltage lamps or traffic signal cabinets. Four active-HIGH push buttons represent pedestrian request inputs. Each button is configured as a rising-edge external interrupt, allowing the firmware to capture requests without polling delays.

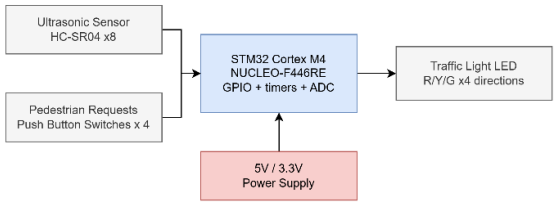

Fig. 1. Embedded system architecture of the proposed adaptive traffic controller.

B. Hardware Methodology

The pin mapping is organized by direction. The firmware defines four directions, DIR_N, DIR_E, DIR_S, and DIR_W, and arrays of GPIO structures for red, yellow, and green LEDs. The red outputs are mapped to PA7, PB6, PA8, and PB5; yellow outputs to PA6, PC7, PB10, and PB3; and green outputs to PA5, PA9, PB4, and PA10. The shared ultrasonic trigger line uses PB0. Each direction has a near and far echo input, and each button maps to an EXTI-capable pin. This array-based organization is important because the same

functions can operate on any direction by index, reducing repetitive code and making the circular algorithm simple.

The prototype uses a shared trigger line for all HC-SR04 modules. A shared trigger reduces GPIO count, but it increases the need for careful measurement sequencing because acoustic crosstalk and residual echoes can corrupt readings. The firmware mitigates this by reading each echo channel using median-of-three samples and introducing a 60 ms settle delay between the near and far readings. The project uses a 30 ms timeout for echo measurements to prevent the program from blocking indefinitely if no echo arrives.

The configured distance threshold is 3 cm for all directions, and readings below a 2 cm noise floor are ignored. This short threshold is appropriate for a tabletop demonstration in which a vehicle is represented by an object placed very close to the sensor. In a real road installation, the same software structure would remain useful, but the sensors, thresholds, and mounting geometry would need to be redesigned for lane-scale distances, weather exposure, and vehicle reflectivity.

TABLE I. HARDWARE RESOURCES AND THEIR ROLE IN THE PROTOTYPE.

Subsystem	Prototype Implementation	Purpose
Controller	STM32 Nucleo-F446RE	Executes local adaptive logic and manages GPIO, EXTI, HAL tick and DWT timing
Vehicle sensing	8 HC-SR04 ultrasonic sensor (two per direction)	Classifies each approach as 0, 1 or 2 blocked sensors
Pedestrian input	4 active high buttons	Sets pending request flag through external interrupts
Traffic Outputs	4 LED traffic light modules	Shows red, yellow and green phases for each approach
Timing	HAL tick and Cortex-M DWT	Provides millisecond phase timing and microsecond ultrasonic timing

C. Software Methodology

The firmware is written in C using STM32 HAL functions. The initialization sequence calls HAL_Init(), configures the system clock, enables the Data Watchpoint and Trace (DWT) cycle counter, and initializes all GPIO pins. The DWT cycle counter provides a convenient microsecond time base because it counts CPU cycles; Arm documentation identifies CYCCNT as the DWT clock cycle counter [9]. The code computes cycles_per_us from the HCLK frequency and converts cycle counts into microseconds for ultrasonic pulse timing.

The ultrasonic function read_one_echo() performs the standard trigger and echo measurement sequence. It pulls the trigger low, waits 2 microseconds, drives the trigger high for approximately 11 microseconds, and then waits for the echo line to rise. The pulse width is measured using the DWT-based micros() function. Distance is computed using the common HC-SR04 conversion in which centimeters are approximated by echo time in microseconds divided by 58. If either the rising edge or the falling edge times out, the function returns 0, which the higher-level logic treats as invalid rather than as a blocked lane.

The read_echo_median() function takes three consecutive readings and returns their median. This is a simple but effective robust filtering method. A single spurious pulse can be rejected if the other two readings are consistent. The read_sensor_pair() function then reads the near sensor, waits for acoustic settling, reads the far sensor, and counts how many readings fall between the noise floor and the threshold. The result is an occupancy class from 0 to 2.

Pedestrian inputs are handled through EXTI interrupt handlers. The callback searches the button array for the pin that triggered the interrupt and sets the corresponding ped_pending flag. The main state machine later checks this flag during the direction's red phase. This separation avoids running long timing sequences inside an interrupt routine and preserves predictable state-machine execution.

TABLE II. PRINCIPAL FIRMWARE CONSTANTS USED IN THE ADAPTIVE CONTROLLER.

Parameter	Firmware value	Design role
Green, no traffic	2s	Avoid wasting a full cycle on an empty approach
Green, low traffic	5s	Provides moderate service for one blocked sensor.
Green, heavy traffic	10s	Extends service when near and far sensors are blocked
Yellow Clearance	3s	Visible transition before returning to red.
All-red gap	2s	Clearance buffer before granting green
Echo timeout	30ms	Prevents indefinite blocking during ranging
Settle delay	60ms	Reduces acoustic crosstalk between readings

D. Adaptive Control Algorithm

The main loop implements a circular state machine. It begins with all approaches red, samples sensors, optionally performs a pedestrian acknowledgement sequence, waits for the red gap, executes a green phase for the current direction, executes a yellow phase, and advances to the next direction. The order is north, east, south, west, and back to north. This ordering guarantees that every direction receives service even if one approach continuously reports heavy traffic.

The adaptive element is the green_duration() function. If no sensor in the selected pair is blocked, the controller grants a 2 s green. If one sensor is blocked, the green becomes 5 s. If both near and far sensors are blocked, the green becomes 10 s. These values match the project objective of short service for empty lanes and longer service for higher density. The result is a bounded priority system rather than an unbounded priority queue. Heavy traffic receives more time, but it cannot permanently remove service from other directions.

The firmware samples a variable named sense_dir, computed as the direction opposite the current green direction. This reflects the physical arrangement used in the prototype, where sensors were mounted to observe the opposite approach relative to the LED direction. In a production design, this mapping would be defined in a configuration table during installation so that logical signal phases always correspond to the correct detector channels.

Pedestrian requests are served during the next safe red phase of the requested direction. The implemented sequence blinks yellow for 2 s to acknowledge the request, then blinks.

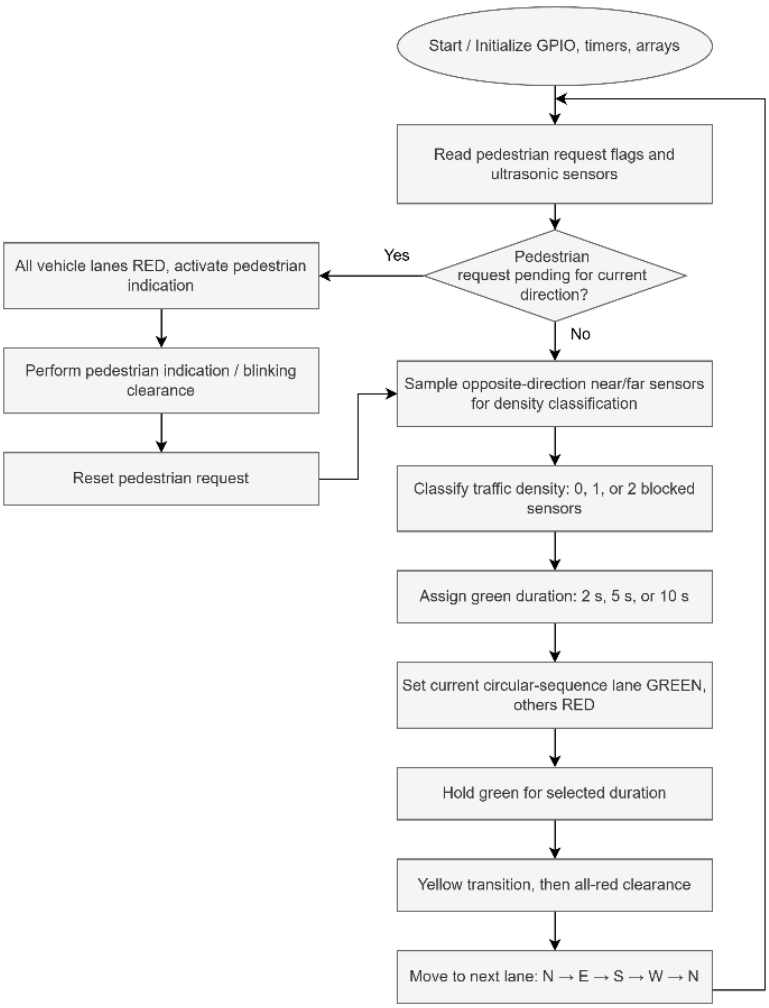

Fig. 2. Flowchart of the adaptive traffic control algorithm.

red for 2 s to represent a pedestrian walk window, and then returns to steady red for the normal red gap before green. This provides visibility without suddenly interrupting a green phase. The design is intentionally conservative because the prototype is focused on demonstrating safe event integration rather than optimizing pedestrian delay

E. Algorithm Complexity and Safety Properties

The algorithm has linear complexity with respect to the number of directions. In the current design, there are four directions and two sensors per direction. If the controller were extended to N approaches, the sensor memory and per-cycle reading time would scale with 2N sensors, while the state transition logic would remain a simple circular loop. The green duration decision itself is constant time because it maps a small integer occupancy count to one of three fixed values.

The main safety property is mutual exclusion: only one direction is ever set to green. The all_red() helper explicitly sets every direction to red before sensor sampling and pedestrian handling. During the green phase, the code turns off red and yellow only for the active direction and blinks its green output. During the yellow phase, only the active direction displays yellow. Other directions remain red throughout. This is a prototype-level version of the interlock principle used in real signal controllers, although a commercial product would also require hardware fail-safe outputs, watchdog supervision, and compliance testing.

F. Firmware Implementation Notes

The firmware was structured to keep hardware-dependent definitions near the top of the source file and reusable logic in helper functions. This is important for maintainability because traffic controller behavior depends on correct mapping between physical wiring and logical phase names. The GpioPin structure stores a port and pin number, and the LED, echo, and button arrays use the same directional index convention. If the hardware wiring is revised, the table entries can be changed without rewriting the state machine. The final source also documents a button remapping step, which is a common integration issue when the physical prototype is wired after the logical design has already been drafted.

A simplified pseudocode view of the implemented algorithm is useful for validation. At each iteration, the controller sets all lights red, selects the detector pair associated with the current direction, filters the ultrasonic measurements, converts the resulting blocked count into a green duration,

serves any pending pedestrian flag, waits the all-red gap, blinks green for the selected duration, displays yellow for the clearance interval, and increments the direction modulo four. The loop has no dynamic memory allocation, no recursion, and no dependence on operating-system services. These properties make the firmware suitable for a small bare-metal microcontroller and simplify worst-case timing analysis.

The source code also highlights practical firmware risks. First, delay-based phase timing is easy to understand but limits background processing. Second, a shared ultrasonic trigger line saves pins but increases dependence on careful sequencing. Third, constants and comments must be kept synchronized: the executable green-duration macros implement 2 s, 5 s, and 10 s, while an older header comment still mentions a previous 5 s, 7 s, and 9 s mapping. This should be corrected before formal release because safety-related firmware documentation must match executable behavior. Finally, the prototype uses software-only light interlocking; a product should add hardware conflict monitoring and a watchdog-controlled fail-red state.

V. System Testing and Validation

Testing was organized to verify the main requirements of the smart adaptive traffic light system: sensor-based traffic classification, adaptive green-light duration, continuous loop operation, fairness among four directions, pedestrian request handling, and safe signal transition. The assignment requires the prototype to adapt traffic light durations based on changing sensor input, prioritize higher traffic density, and prevent any direction from being blocked for an extended period. Therefore, the validation process was designed to test both the functional behavior and the timing behavior of the implemented STM32 controller.

A. Test Setup

The prototype was tested under controlled bench conditions using the STM32 Nucleo-F446RE board, eight HC-SR04 ultrasonic sensors, four pedestrian push buttons, and four red-yellow-green LED traffic light modules. Representative objects were placed in front of the ultrasonic sensors to simulate vehicle presence. Each direction used a near and far ultrasonic sensor pair, allowing the system to classify the traffic condition as empty, low density, or heavy density. The programmed timing constants were 2 s green for no traffic, 5 s green for low traffic, 10 s green for heavy traffic, 3 s yellow clearance, and 2 s all-red clearance.

Timing validation was performed using recorded video playback and manual stopwatch measurement. Each major test case was repeated several times to reduce the possibility of judging the system from a single successful demonstration. The observed LED behavior was then compared with the expected firmware logic. Since the prototype was tested on a tabletop model rather than a real road junction, the validation is reported as prototype-level functional testing rather than field traffic performance evaluation.

B. Sensor Classification Test

The first test verified whether the ultrasonic sensor pairs could correctly classify the simulated traffic condition. For the empty-lane condition, no object was placed within the detection threshold. For the low-density condition, only one sensor in a direction was blocked. For the heavy-density condition, both the near and far sensors were blocked. The expected output was an occupancy value of 0, 1, or 2 respectively.

The test confirmed that the sensor-pair logic correctly converted vehicle presence into three traffic-density levels. The median-of-three reading method helped reduce the effect of unstable ultrasonic readings. The 60 ms settle delay between sensor readings also reduced the likelihood of false triggering caused by acoustic crosstalk between the near and far sensors.

C. Adaptive Timing Test

The second test verified whether the controller selected the correct green-light duration based on the detected traffic density. When no sensor was blocked, the green phase was expected to last approximately 2 s. When one sensor was blocked, the green phase was expected to last approximately 5 s. When both sensors were blocked, the green phase was expected to last approximately 10 s.

The observed LED behavior followed the intended adaptive timing logic. Empty approaches received the shortest green interval, while heavier traffic conditions received longer green intervals. This confirms that the STM32 controller did not simply operate as a fixed-time traffic light, but adapted its output timing according to real-time sensor input.

D. Pedestrian Request Test

The pedestrian request function was tested by pressing the pedestrian button during different parts of the traffic-light cycle. The purpose was to verify that a pedestrian request could be captured immediately without interrupting an active vehicle green phase unsafely. When a button was pressed, the interrupt service routine stored the request as a pending flag. The request was then served only when the corresponding direction entered a safe red phase.

The test confirmed that pedestrian requests did not immediately terminate the active green light. Instead, the system acknowledged the request during the next appropriate red phase using the programmed blinking sequence. This behavior demonstrates that the firmware separates event detection from event execution, which improves safety and preserves deterministic traffic-light operation.

E. Fairness and Starvation Test

The fairness test was performed by keeping one direction in a heavy-traffic condition over repeated cycles while observing whether the other directions still received green service. The expected behavior was that the heavy-traffic direction would receive a longer green duration, but would not permanently dominate the junction.

The test confirmed that the circular service order continued to operate in the sequence north, east, south, and west. Since the direction index is incremented after each phase, every direction eventually receives a green phase. Therefore, the adaptive logic improves responsiveness to traffic density while still preventing starvation.

F. Safety Transition Test

The safety transition test verified the yellow and all-red clearance intervals between direction changes. After each green phase, the active direction changed to yellow for approximately 3 s before all directions entered a red state for approximately 2 s. The test also verified that no two directions showed green at the same time.

TABLE III. SYSTEM VALIDATION TEST MATRIX.

Test case	Input condition	Expected response	Validation method	Result
T1: Empty lane	No object within near/far threshold	Occupancy = 0, green = 2 s	Sensor activation and video timing	Pass
T2: Low density	One sensor blocked	Occupancy = 1, green = 5 s	Sensor activation and video timing	Pass
T3: Heavy density	Near and far sensors blocked	Occupancy = 2, green = 10 s	Sensor activation and video timing	Pass
T4: Pedestrian request	Button pressed during active cycle	Request stored and served during safe red phase	Button press observation	Pass
T5: Starvation check	One direction remains heavy for repeated cycles	All directions still receive service	Full-cycle observation	Pass
T6: Safety transition	Direction changes after green phase	3 s yellow and 2 s all-red occur before next green	LED sequence observation	Pass
T7: Green conflict check	Normal continuous operation	No two directions green at the same time	Visual inspection	Pass

The observed behavior confirmed that the firmware maintained mutual exclusion between green signals. Only one direction was allowed to display green during a vehicle phase, while all other directions remained red. The yellow and all-red intervals provided a visible clearance buffer before the next direction received green.

VI. RESULTS

The prototype achieved the main functional requirements of the assignment. The STM32-based controller successfully used sensor input to classify traffic density and adjust the green-light duration accordingly. When no object was detected within the threshold distance, the selected green duration was 2 s. When one sensor was blocked, the selected green duration increased to 5 s. When both the near and far sensors were blocked, the selected green duration increased to 10 s. This confirms that the prototype operated as an adaptive traffic light system rather than a fixed-time signal controller.

The sensor classification results showed that the near/far ultrasonic sensor arrangement was sufficient for a tabletop demonstration of three traffic-density levels. Although the system does not measure exact queue length, it provides enough information for simple adaptive timing. The median-of-three filtering method improved the stability of the readings by reducing the effect of single abnormal measurements.

The timing results showed that the programmed green-light durations were reflected in the visible LED sequence. The yellow interval and all-red gap were also observed during each transition between directions. These safety buffers are important because they prevent the controller from switching directly from one green phase to another. Throughout testing, no case was observed in which two directions displayed green at the same time.

The pedestrian request test showed that the external interrupt mechanism successfully captured button presses. The request was not executed immediately during an active vehicle green phase. Instead, it was stored as a pending request and served during the corresponding direction's safe red phase. This result confirms that asynchronous pedestrian input can be integrated without disrupting the deterministic vehicle signal sequence.

The starvation test confirmed that the circular phase sequence maintained fairness. Even when one direction remained in a heavy-traffic condition, the controller continued to serve the other three directions. Heavy traffic increased the green duration for the affected direction, but it did not change the fixed circular order. Therefore, the system achieved a balance between traffic-density priority and fairness.

A timing comparison can be derived from the firmware constants. If all four approaches are empty, the approximate cycle time is 28 s, calculated from 2 s green, 3 s yellow, and 2 s all-red for each direction. If all four approaches are low density, the cycle time becomes approximately 40 s. If all four approaches are heavy, the cycle time becomes approximately 60 s. This demonstrates the intended adaptive behavior: the controller reduces unnecessary green time under low-demand conditions while extending service when higher traffic density is detected.

TABLE IV. ADAPTIVE CYCLE TIME DERIVED FROM FIRMWARE CONSTANTS.

Condition	Per-direction nominal time	Four-direction cycle estimate
All approaches empty	2 s green + 3 s yellow + 2 s all-red = 7 s	28s
All approaches low density	5 s green + 3 s yellow + 2 s all-red = 10 s	40s
All approaches heavy	10 s green + 3 s yellow + 2 s all-red = 15 s	60s
One pedestrian request	Additional 4 s indication sequence	Adds 4 s to the affected direction

Overall, the results show that the prototype meets the project objective of demonstrating a smart adaptive traffic light system for a four-way junction. The system adapts signal timing based on sensor input, maintains a safe signal sequence, captures pedestrian requests, and prevents

starvation through circular servicing. However, the results are limited to bench-level prototype validation. A real traffic deployment would require more rigorous testing, including calibrated vehicle detection, long-duration reliability testing, environmental testing, and compliance with traffic signal safety standards.

VII. Discussion

A. Design Benefits

The main benefit of the prototype is that it demonstrates adaptive behavior using a low-cost and explainable embedded system. Compared with a fixed timer, the controller avoids allocating long green time to an empty approach. Compared with complex adaptive systems, the controller remains simple enough to understand from source code inspection. This is valuable in safety-related systems because operators and maintainers must be able to predict behavior under failure and edge cases.

The system also has a clear educational value. It integrates GPIO outputs, external interrupts, microsecond timing, sensor filtering, and state-machine design in one project. The same structure can be reused for other embedded control problems in which asynchronous requests must be captured quickly but served only during safe states.

B. Challenges and Limitations

The most significant hardware challenge is ultrasonic sensing reliability. Ultrasonic modules can be affected by object angle, surface material, environmental noise, temperature, and crosstalk. The project mitigates these issues with median filtering, a noise floor, a timeout, and settle delays. However, a real outdoor installation would require weatherproof sensors or alternative detectors such as radar, loop detectors, magnetometers, or camera analytics. The 3 cm threshold used in the prototype is not intended for road-scale deployment.

A second limitation is the blocking software structure. The main loop uses HAL_Delay() and busy-wait timing for phase execution. This approach is acceptable for a simple prototype because the controller has one primary task and a small number of interrupts. A commercial controller would likely use a non-blocking scheduler, hardware timers, watchdog supervision, and explicit safety states. This would allow diagnostics, communications, logging, and fail-safe outputs to run concurrently with phase timing.

A third limitation is that the density model has only three levels. This is intentional for clarity, but it cannot distinguish between two vehicles and a long queue once both sensors are blocked. It also does not estimate arrival rate, turning movements, queue discharge, or downstream spillback. The method is therefore best viewed as a first-stage adaptive controller. Future work could add additional sensors per lane, moving-average occupancy, queue-length estimation, or communication with neighboring intersections.

Finally, the prototype uses LEDs rather than real signal heads. A commercial product would need isolated high-power output drivers, surge protection, manual override, maintenance mode, conflict monitoring, cabinet integration, and compliance with local traffic signal standards. These requirements do not invalidate the algorithm, but they significantly change the hardware design effort.

C. Scalability

The design scales well at the algorithmic level. For a junction with more approaches or turn phases, the circular array structure can be extended by increasing NUM_DIRS and adding phase definitions. The cost of reading sensors scales linearly with the number of sensor pairs. Since the controller makes local decisions without cloud dependency, it can operate even when network connectivity is unavailable. This is useful for rural intersections, campus roads, industrial sites, temporary construction intersections, or pilot smart-city deployments.

Network scalability would require additional design. A single isolated controller can improve local efficiency, but adjacent intersections can still create platoon effects and queues. To scale to a corridor, each controller could share compact state variables such as phase, queue class, and expected discharge with neighboring controllers. This would preserve the edge-computing philosophy while enabling coordination. A more advanced version could use the simple rule-based controller as a safe fallback and enable optimization only when communication is healthy.

D. Manufacturing Cost and Commercialization

The prototype is intentionally low cost. At classroom scale, the major components are the Nucleo development board, eight HC-SR04 modules, four button inputs, LED traffic modules, wires, and a breadboard or small PCB. A reasonable prototype bill of materials is in the tens of U.S. dollars, excluding tools and labor. For production, the Nucleo board would be replaced by a custom PCB containing an STM32F446 or a lower-cost STM32 variant, protected sensor interfaces, power regulation, output drivers, and connectors. The HC-SR04 modules would also be replaced by sealed outdoor sensors or another certified detector technology.

TABLE V. Cost considerations for moving from prototype to product.

Cost item	Prototype estimate	Commercial implication
Controller	Nucleo board or STM32 PCB	Custom PCB reduces unit cost but adds design and certification effort.
Sensors	8 low-cost ultrasonic modules	Outdoor-rated detectors dominate installed sensor cost.
Outputs	LED modules	Real signals require isolated drivers or cabinet interfaces.
Housing/ power	Breadboard or lab supply	Rugged enclosure, surge protection, and power conversion required.
Software	Rule-based firmware	Needs diagnostics, logs, watchdog, update process, and configuration tool.

A commercial version could be positioned as an AdaptiveFlow smart-junction retrofit kit. The product would include a cabinet-mounted edge controller, four or more detector nodes, isolated interfaces to existing signal heads or controller inputs, a configuration tool, and optional connectivity for monitoring. Installation would involve mounting sensors, mapping detector channels to phases, setting thresholds, testing conflict outputs, and validating timing plans with local traffic engineers. The core value proposition is smart-junction behavior without requiring expensive cameras or continuous cloud analytics.

Manufacturing costs depend strongly on ruggedization and certification. The bare electronics could be inexpensive, but the deployed product must include an enclosure, surge protection, connectors, conformal coating, power supply, watchdog circuits, and regulatory testing. A practical commercialization path would therefore begin with private campuses, car parks, logistics yards, and temporary work zones where regulatory barriers are lower. After reliability data are collected, the design could be adapted for municipal pilot programs and integrated with existing traffic cabinets as a detector-assisted timing module rather than as a complete replacement controller.

E. Deployment and Validation Roadmap

Commercial deployment should proceed in stages because an adaptive signal controller affects both safety and public trust. The first stage is laboratory hardening. In this stage, the firmware should be refactored into non-blocking timed states, hardware watchdog recovery should be added, and all comments, constants, and configuration tables should be audited. The sensor subsystem should also be tested against different object surfaces, ambient temperatures, and mounting angles. The result of this stage should be a repeatable bench test plan rather than a one-time demonstration.

The second stage is a controlled private-site pilot. Suitable locations include a campus road, industrial yard, car park entrance, or temporary construction crossing. These sites allow realistic vehicle movement while keeping regulatory complexity lower than a public arterial. During the pilot, the controller should log every detector state, phase decision, pedestrian request, and fault condition. These logs would allow objective comparison between fixed-time and adaptive operation using metrics such as average waiting time, maximum waiting time, number of stops, false detector activations, missed detections, and controller uptime.

The third stage is integration with traffic engineering practice. For a municipal pilot, the device should not initially replace certified conflict monitors or cabinet controllers. A safer path is to operate as an auxiliary detector and timing recommendation module, where existing certified equipment retains final authority over the signal heads. Once reliability and safety evidence are established, the edge controller could be packaged as a certified retrofit product with configuration software, remote health reporting, encrypted firmware updates, and documented fail-safe behavior.

TABLE VI. SUGGESTED ROADMAP FOR COMMERCIAL DEPLOYMENT AND VALIDATION.

Stage	Main activities	Success evidence
Laboratory hardening	Non-blocking firmware, watchdog, documentation audit, sensor calibration	Repeatable tests and consistent timing under fault conditions.
Private-site pilot	Install at campus or industrial junction; log detector and phase states	Measured reduction in delay without missed pedestrian or safety events.
Municipal integration	Interface with certified cabinet equipment and traffic engineering review	Compliance documentation, fail-safe validation, and maintainable configuration tool.

VIII. CONCLUSION

This paper presented a complete embedded prototype for smart adaptive traffic management at a four-way junction. The system uses an STM32 Nucleo-F446RE, eight ultrasonic sensors, four pedestrian buttons, and LED signal modules to demonstrate adaptive green-time selection, bounded circular service, safe yellow and all-red buffers, and deferred pedestrian request handling. The firmware implements microsecond ultrasonic ranging with the Cortex-M4 DWT cycle counter, median-of-three filtering, EXTI-based button capture, and a deterministic traffic state machine.

The results show that a low-cost edge controller can provide meaningful adaptive behavior without cameras, cloud processing, or complex optimization. Empty approaches receive short green time, occupied approaches receive longer service, and every direction remains guaranteed service through the circular sequence. The system is therefore scalable in concept, although real deployment would require additional sensing robustness, non-blocking firmware, fail-safe hardware, certification, and integration with traffic signal infrastructure.

From a commercial perspective, the design is best viewed as a foundation for a low-cost smart-junction retrofit solution. Its main advantages are simplicity, explainability, low hardware cost, and local operation. Potential early markets include campuses, industrial roads, car parks, and temporary traffic control sites. With ruggedized sensors, isolated outputs, configuration software, diagnostics, and compliance testing, the system could evolve into a deployable product that provides adaptive traffic behavior at a fraction of the complexity of camera-based or cloud-dependent systems.

IX. CODE AVAILABILITY

The source code used for this prototype is available at: https://github.com/21f3003256/Smart-Traffic-Light-Controller/tree/main

X. REFERENCES

[1] P. Koonce *et al*., *Traffic Signal Timing Manual*, FHWA-HOP-08-024. Washington, DC, USA: Federal Highway Administration, 2008.

[2] L. A. Rodegerdts *et al*., *Signalized Intersections: Informational Guide*, FHWA-HRT-04-091. McLean, VA, USA: Federal Highway Administration, Aug. 2004.

[3] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, *SCOOT—A Traffic Responsive Method of Coordinating Signals*, Laboratory Report 1014. Crowthorne, U.K.: Transport and Road Research Laboratory, 1981.

[4] S. F. Smith, G. J. Barlow, X.-F. Xie, and Z. B. Rubinstein, "SURTRAC: Scalable urban traffic control," in *Proc. Transportation Research Board 92nd Annual Meeting*, Washington, DC, USA, 2013.

[5] J. Gao, Y. Shen, J. Liu, M. Ito, and N. Shiratori, "Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network," *arXiv preprint arXiv:1705.02755*, 2017.

[6] A. Shams, A. M. T. Emtenan, and C. M. Day, "A taxonomy of adaptive traffic signal control," *arXiv preprint arXiv:2308.01952*, 2023.

[7] STMicroelectronics, *STM32F446xC/E Arm Cortex-M4 32-bit MCU+FPU*, DS10693, Datasheet, 2021.

[8] STMicroelectronics, *STM32 Nucleo-64 Boards (MB1136)*, UM1724, User Manual, 2020.

[9] Arm Ltd., *Arm Cortex-M4 Processor Technical Reference Manual*. Cambridge, U.K.: Arm Ltd., 2010.

ELECFreaks, *Ultrasonic Ranging Module HC-SR04*, Datasheet.

[10] M. A. S. Kamal, J. Imura, A. Ohata, T. Hayakawa, and K. Aihara, "Traffic signal control in an MPC framework using mixed integer programming," in *Proc. IFAC Symposium on Advances in Automotive Control*, Tokyo, Japan, 2013, pp. 641–646.

[11] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, London, U.K., 2018, pp. 2496–2505.